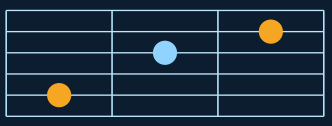
COMPÁS A COMPÁS



Diseño técnico del software

Arquitectura, datos, paginación y operación local

Autor del proyecto: Jesús Quevedo Rodríguez

Aplicación 0.5.0 · 17/10/2026

Repositorio: a01653/Compas-a-compas

Índice

Objeto y alcance 3

Arquitectura y ejecución 3
- Componentes . . . 3
- Flujo de datos . . . 4

Interfaz compacta y contexto de compás 4

Paleta editable 5

Modelo y contrato JSON 5
- Entidades . . . 5
- Contexto tonal por posición . . . 6
- Identidad y validación . . . 6
- Textos e importación de listas . . . 7

Estado, edición y persistencia 7
- Historial de acciones . . . 7

Geometría y notación 8
- Mástil y tarjetas . . . 8
- Repeticiones y saltos . . . 9

Medición y paginación 9
- Dimensiones vigentes . . . 9
- Algoritmo . . . 9
- Cambios concurrentes de interfaz . . . 10

Exportación y tratamiento de texto 10

Servidor y lanzador local 10

Comprobaciones y límites de la evidencia 11

Mantenimiento y documentación 12

Aplicación 0.5.0 · Esquema de canción 1 · Edición del DTS: 17/10/2026. Autor del proyecto: Jesús Quevedo Rodríguez.

Objeto y alcance

Compás a compás es un editor local de hojas de acordes de guitarra. Organiza una canción en compases, admite varios acordes por compás y varias digitaciones por acorde. Genera una vista previa paginada y una salida HTML autónoma para impresión A4 horizontal.

Este documento describe la versión 0.5.0, con interfaz compacta, textos sin bloques duplicados, historial de acciones, marcas por compás, paleta editable y tonalidades por posición. Incluye el lanzador de Windows. Los requisitos originales permanecen en `docs/REQUISITOS.md`; el historial de implementación y sus comprobaciones está en `docs/ESTADO.md`. Este DTS describe el comportamiento vigente, por encima de las medidas históricas de fases anteriores.

No se incluyen reproducción, OCR, transposición, análisis armónico, generación automática de posiciones ni sincronización en la nube. Las marcas musicales son notación escrita: no hay un motor que ejecute o expanda el recorrido.

Arquitectura y ejecución

Componentes

La aplicación usa HTML, CSS y módulos JavaScript nativos. No tiene framework, backend de negocio, base de datos ni dependencias npm de ejecución. Node.js 18 o superior sirve los archivos al navegador mediante HTTP local.

Capa	Archivos principales	Responsabilidad
Entrada y estado	`src/main.js`, `src/store.js`	Inicialización, canción única, selección y notificaciones de cambio
Modelo	`src/model/`	Entidades, IDs, validación, serialización, listas y referencias musicales
Geometría	`src/geometry/fretboard.js`	Intervalo visible y coordenadas del mástil
Renderizado	`src/render/`	Tarjetas, cabeceras, compases y marcas mediante DOM y SVG
Paginación	`src/layout/`	Fuentes, unidades, medición DOM y reparto en filas y hojas
Interfaz	`src/ui/`	Edición, archivos, diálogos, zoom, vista previa y exportación
Recuperación	`src/persist/recovery.js`	Copia local de cambios pendientes

Capa	Archivos principales	Responsabilidad
Salida	`src/export/document.js`	Ensamblado del HTML autónomo
Arranque	`Abrir-Compas.cmd`, `scripts/launch.mjs`, `scripts/serve.mjs`	Lanzador y servidor estático local

Los estilos de interfaz están en `styles/app.css`; la hoja usa estilos compartidos de `styles/sheet.css`, tokens y exportación. Las fuentes Caveat Brush y Gochi Hand están incluidas en `references/fonts.css`, sin descarga de fuentes remotas. El icono de la aplicación es `assets/icon.svg`.

Flujo de datos

1. El usuario modifica la canción mediante un controlador de interfaz.
2. El controlador notifica el tipo de cambio al almacén.
3. Los paneles afectados actualizan su presentación.
4. Si cambió el contenido, el paginador obtiene una copia de la canción, carga las fuentes y mide el mismo renderizador que se usará en la salida.
5. El reparto puro decide filas, fragmentos y páginas; la vista previa muestra el último resultado vigente.
6. Exportar espera ese resultado, comprueba que sigue vigente y construye el HTML desde su copia medida.

El JSON conserva el contenido editable. Coordenadas, filas, número de páginas y zoom son datos derivados y no se guardan como parte de la canción.

Interfaz compacta y contexto de compás

El editor usa dos columnas: controles de compás, acorde y posición a la izquierda, y hoja a la derecha. Los selectores permiten recorrer listas extensas sin aumentar la altura del formulario. Digitación, grados y textos están siempre visibles en dos secciones apiladas, sin controles de pestaña. Nombre, tonalidad y versión del documento se editan en un diálogo. Exportar agrupa HTML e impresión. Los iconos tienen nombre accesible y los menús se cierran con Escape o al pulsar fuera.

El acceso a Marcas pertenece al bloque Compás y anuncia su etiqueta en el nombre accesible. El panel filtra símbolos y saltos por el compás donde están escritos, y repeticiones y casillas por todos los compases del tramo, incluidos sus extremos. `src/model/marksContext.js` obtiene esa vista sin modificar la estructura. Un extremo desaparecido sigue siendo localizable desde el que existe; todas las marcas permanecen accesibles en la vista global.

Cambiar de compás reinicia el borrador del formulario con ese compás como punto de partida y termina la edición local no guardada; no cambia marcas persistidas. Elegir otro acorde o posición dentro del mismo compás conserva el borrador. Las marcas existentes se editan usando sus IDs y sus destinos originales, aunque el tramo empiece en otro compás. Ver todas las marcas cambia solo el filtro; los avisos globales continúan visibles en ambas vistas.

`ui/workspace.js` coordina los diálogos y el contador, y `ui/marks.js` gestiona el contexto y el formulario. `styles/workspace.css` adapta el espacio a la altura disponible. El móvil usa paneles apilados; avisos extensos, formularios largos y varias hojas mantienen desplazamiento cuando se necesita. Las nuevas cabeceras de tarjeta cambian la composición de la hoja, compartida con la exportación. La geometría del mástil no cambia.

Paleta editable

`src/model/palette.js` define la resolución de colores, sus valores originales y el contraste del texto. El campo opcional `song.palette` contiene `families`, `degrees` y `note`. La paleta se valida con el mismo contrato en el esquema JSON y el validador manual. Los colores son exclusivamente hexadecimales de seis cifras; no se aceptan valores CSS arbitrarios. Las etiquetas personalizadas tienen entre 1 y 32 caracteres y excluyen claves reservadas de objetos JavaScript.

El esquema 1 se amplía de forma opcional: los archivos anteriores siguen válidos y se guardan sin cambios si no se personalizan. Un archivo con `palette` requiere 0.3.0 o posterior; los lectores anteriores rechazan el campo desconocido. No se presenta esa compatibilidad como lectura posible en ambos sentidos.

El panel `ui/colors.js` ofrece familias, grados naturales y alterados del 1 al 13, etiquetas personalizadas y notas sin grado. Solo modifica la paleta de la canción; las notas, IDs, textos y destinos se conservan. La normalización acepta ♭/♯ como b/#. Cada etiqueta puede personalizarse de forma independiente. La restauración individual retira su entrada; la restauración completa, tras confirmar, elimina `palette`.

El contexto de hoja pasa la paleta a tarjetas y mástiles, compartiendo medición, vista previa y exportación. Los textos y trazos interiores se adaptan al fondo; los títulos exteriores mantienen su color sobre el papel. Los colores de notas se serializan en SVG y los fondos como variables CSS locales. La impresión usa el mismo documento. No se modifican geometría ni digitaciones al cambiar un color. Las notas al aire conservan el dibujo de círculo o punto según tengan grado.

Modelo y contrato JSON

El contrato formal es `schemas/song.schema.json`. `src/model/validate.js` implementa la validación utilizada por la aplicación sin cargar una biblioteca externa. `src/model/serialize.js` convierte entre objetos y JSON con sangría de dos espacios y salto final.

Entidades

Entidad	Contenido
Canción	`schemaVersion`, `title`, `key`, `measures`, `structure`; `documentVersion`, `keyFamily`, `showTonalities` y `palette` opcionales
Compás	`id`, `label` y lista ordenada `chordEvents`
Acorde	`id`, `chordName`, lista `voicings`; familia y textos opcionales
Posición	`id`, `frets` de seis valores; grados de cuerda, textos, `voicingDegree` y `tonality` opcionales
Estructura	Listas `repeats`, `endings`, `markers` y `jumps`

Los trastes se almacenan de sexta a primera cuerda: entero no negativo o `null` para una cuerda muda. Los grados, cuando existen, son seis textos en el mismo orden. La versión de esquema es numérica y vale 1; la versión de documento es texto libre del usuario. Ninguna de ellas es la versión 0.5.0 del programa.

Ejemplo mínimo válido:

```
{
  "schemaVersion": 1,
  "title": "Ejemplo",
  "key": "Do menor",
  "measures": [{
    "id": "m1", "label": "1",
    "chordEvents": [{
      "id": "e1", "chordName": "Cm7",
      "voicings": [{"id": "v1", "frets": [null, 3, 5, 3, 4, 3]}]
    }]
  }],
  "structure": {"repeats": [], "endings": [], "markers": [], "jumps": []}
}
```

`examples/cancion-demo.json` amplía el ejemplo con dos posiciones de un acorde, dos acordes en otro compás, cuatro trastes visibles, repeticiones, casillas y saltos. Es una demostración, no una transcripción musical.

Contexto tonal por posición

`src/model/tonality.js` resuelve el contexto según el orden escrito. La tonalidad general inicia el recorrido con `keyFamily`, o con `other` si falta. Sin tonalidad general ni cambio previo no hay contexto. Cada objeto `voicing.tonality`, con `text` visible y `family` válida, establece un cambio; su ausencia hereda el contexto anterior. El contexto se deriva, no se serializa por duplicado, y no expande las repeticiones.

`voicingDegree` es texto manual de la esquina superior derecha, distinto de los seis `degrees` de cuerda. Vaciar ese campo o la tonalidad retira su propiedad. Ambos pertenecen a una posición; el alcance de todas las posiciones solo afecta al título y anotación existentes. Elegir una calidad con tonalidad vacía es un borrador del control y no guarda un cambio invisible.

`showTonalities` es un booleano opcional, visible por defecto. Oculta las franjas en todas las hojas sin borrar datos y conserva la tonalidad de cabecera. La casilla y los campos forman parte del historial y de la recuperación. La familia de la tonalidad usa la paleta de canción, independientemente de la familia del acorde.

Estos campos amplían el esquema 1 de forma opcional. Los archivos anteriores siguen válidos sin migración ni modificación silenciosa; un archivo que incluya campos nuevos necesita 0.5.0 o posterior. Los validadores manual y JSON Schema rechazan tipos, familias y propiedades desconocidos.

Identidad y validación

Los identificadores son estables y únicos en el documento. La etiqueta del compás es independiente de su posición en el array. Mover o renombrar un compás no cambia sus referencias. Duplicar crea IDs nuevos. La fábrica reserva también identificadores de referencias pendientes para evitar que una entidad posterior capture un destino borrado.

La apertura valida antes de sustituir el estado actual. JSON mal formado, tipos incorrectos, campos desconocidos, IDs duplicados o una versión de esquema no admitida producen errores y dejan intacta la canción abierta. No se eliminan silenciosamente campos desconocidos.

La validación distingue la forma del archivo de sus referencias semánticas. Un destino musical ausente puede conservarse con aviso; no se reasigna por índice ni por una etiqueta parecida. Al exportar referencias pendientes se pide confirmación.

Textos e importación de listas

El acorde puede definir `aboveText` e `insideText`. En una posición, la ausencia del campo significa heredar; una cadena propia sustituye el texto y la cadena vacía lo oculta. Se mantiene la diferencia al guardar y abrir. El formulario 0.4.0 muestra solo el texto efectivo, sin dos bloques superpuestos de controles. `setCardText` modifica el valor propio de la posición; el modo de todas las posiciones cambia el valor común y retira las excepciones solo de ese campo. Vaciar el campo individual guarda cadena vacía; vaciarlo para todas retira el común y sus excepciones. Cambiar de posición reinicia el destino de edición a individual.

El importador usa una línea por compás: `etiqueta | acorde @ digitación`. Separa eventos por `;` y posiciones mediante `/` tras `@`. Conserva nombres como `C/E`. Genera una previsualización y añade el lote al final únicamente si todas las líneas son válidas. No calcula duraciones ni grados.

Estado, edición y persistencia

`createStore` mantiene una canción mutable y una selección por IDs. Guarda además el nombre de archivo, el zoom, la firma del último guardado y el indicador de cambios pendientes. La firma se obtiene con `JSON.stringify`; `markSaved` actualiza la referencia de guardado.

Los tipos de cambio son `text`, `structure`, `document`, `history`, `selection`, `zoom` y `status`. Un cambio de texto actualiza la hoja sin reconstruir todos los controles, para conservar el foco. Cambios estructurales o de documento normalizan la selección a objetos existentes. Selección, zoom y estado de guardado no provocan una nueva paginación.

Nuevo y Abrir gestionan los cambios pendientes antes de reemplazar el documento. Guardar usa el selector de archivos cuando el navegador lo permite y, como alternativa, una descarga con Blob. Cancelaciones y errores se muestran sin sustituir la canción por un archivo fallido. Exportar HTML e imprimir no marcan el JSON como guardado.

La recuperación usa la clave `compas-a-compas:recuperacion:v1` de `localStorage`, con fecha de guardado y canción serializada. Se ofrece tras recargar cuando existe una copia válida. Guardar, abrir o crear una canción retira la recuperación anterior. Un fallo del almacenamiento no impide editar; el JSON sigue siendo la copia persistente elegida por el usuario.

`localStorage` pertenece al origen: cambiar de navegador, perfil o puerto no comparte esa recuperación. Se usa también el aviso de salida con cambios pendientes cuando el navegador lo admite. No se sobrescribe automáticamente un archivo con nombre.

Historial de acciones

`createStore` conserva estados anterior y posterior de cada cambio de canción, con su selección por IDs. `emit` registra cambios reales de texto o estructura; selección, zoom y guardado no añaden pasos. `ui/history.js` agrupa las entradas mientras el mismo campo conserva el foco. Un cambio estructural, guardar o cambiar de selección cierra el grupo. Deshacer y Rehacer restauran una

copia de la canción, normalizan la selección, comparan con el último guardado y notifican `history`; se refrescan los formularios, el paginador y la recuperación. Los borradores de marcas se reinician al restaurar para no conservar una edición de un objeto que ya no exista.

Hay hasta 100 pasos, con un presupuesto aproximado de 8 MiB para las cadenas serializadas contadas de forma conservadora. Se mantiene al menos el último paso aunque una canción grande supere ese presupuesto. Abrir, crear o recuperar otro documento limpia las dos pilas; guardar las conserva y actualiza la firma de guardado. Una edición nueva descarta la pila de rehacer. El historial pertenece a la sesión y no se guarda en el JSON ni en localStorage.

Los botones están en cabecera y dentro de los tres diálogos de edición. Ctrl+Z, Ctrl+Y y Ctrl+Mayús+Z operan fuera de campos; dentro se conserva el deshacer nativo. Los atajos globales no actúan sobre un diálogo de confirmación o importación, para no invalidar referencias mientras una operación espera respuesta.

Geometría y notación

Mástil y tarjetas

`layoutFretboard` es una función pura. Recorre las seis notas y calcula el intervalo visible. Con cuerdas al aire empieza en el traste 1; de lo contrario comienza en el menor traste positivo. Si no hay notas pulsadas, mantiene el intervalo mínimo.

Parámetro de la versión 0.1.3	Valor
Ancho por traste	28 px CSS
Separación entre cuerdas	17 px CSS
Zona de cuerdas al aire y mudas	26 px CSS
Relleno superior / inferior	10 / 18 px CSS
Intervalo visible mínimo / máximo	3 / 32 trastes
Rejilla de tres trastes, sin zona exterior	84 × 85 px CSS

El ancho SVG es `26 + 28 × númeroDeTrastes + 4`; su altura es `10 + 5 × 17 + 18`, es decir, 113 px. La primera cuerda se dibuja arriba y la sexta abajo. Los puntos se centran en el intervalo entre líneas de traste. El dato almacenado conserva el orden sexta–primera.

El máximo de 32 es un límite conservador de visualización, no el número de traste más alto permitido: una posición alrededor del traste 40 puede dibujarse si su intervalo es pequeño. Si el intervalo supera el límite, no se crean estructuras proporcionales al valor recibido; se muestra un aviso y se conserva la digitación. La salida queda bloqueada.

Los grados son manuales. Los reconocidos emplean la paleta de la plantilla; los textos largos se abrevian en los círculos y conservan una leyenda por cuerda. Las cuerdas mudas no dibujan grados. La familia es una elección visual, no un análisis de la digitación.

Repeticiones y saltos

Repeticiones y casillas referencian compases por ID. `times` es el total de pasadas; las casillas contienen sus pasadas y una repetición opcional. Las marcas Segno, Coda y Fine referencian un compás y un borde `before` o `after`.

Los siete saltos admitidos son `dc`, `ds`, `dc-al-coda`, `ds-al-coda`, `dc-al-fine`, `ds-al-fine` y `to-coda`. Referencian las marcas de destino correspondientes. El panel permite elegir destinos con etiquetas legibles, manteniendo los IDs internamente.

El borrado de un compás presenta las consecuencias para la estructura y permite cancelar. Las referencias que queden pendientes se conservan y se señalan. `marksProjection.js` transforma la estructura para dibujar barras, corchetes y símbolos. No interpreta indicaciones escritas como texto libre ni comprueba que el recorrido termine.

Medición y paginación

Dimensiones vigentes

La fuente de medidas de página es `src/layout/spec.js`. Cada hoja recibe sus variables CSS desde `pageStyleVars`, tanto en el editor como en medición y exportación.

Parámetro	Valor
Papel	A4 horizontal, 297 × 210 mm
Márgenes interiores	5 mm
Área útil	287 × 200 mm
Conversión de unidades	96 / 25,4 px CSS por mm
Separación de compases / filas / cabecera	6 / 12 / 14 px CSS
Reserva de reparto	1 px CSS
Separación visual entre hojas	24 px CSS al 100 %

Cada hoja incluye título seguido de tonalidad y numeración actual/total arriba a la derecha. La versión del documento no se añade al pie de impresión.

Algoritmo

`atoms.js` representa unidades de contenido; una tarjeta y su título forman una pieza que no se divide. `measurer.js` renderiza los mismos nodos de `render/sheet.js` fuera de pantalla y lee sus dimensiones reales, después de cargar las fuentes con una muestra que contiene el texto de la canción.

`computeLayout` coordina el medidor y `paginate`, que decide el reparto sin acceder al DOM. No fija cuatro compases por fila. Primero intenta colocar el compás completo y, si no cabe ni en una fila propia, lo divide entre tarjetas. Los fragmentos mantienen etiqueta y orden e indican continuación. Inicio y final de repetición aparecen solo en el primer y último fragmento; casillas y rangos pueden continuar entre filas y páginas.

Las filas visibles distribuyen el ancho sobrante entre compases mediante flex. Dentro de cada fragmento, una rejilla con subgrid comparte tres filas: títulos enmarcados, franjas de tonalidad

y zona de número/tarjeta. Los eventos abarcan las columnas de sus posiciones. Esto mantiene los títulos fuera de la línea del compás aunque una tarjeta tenga anotación o leyenda más alta. Las pistas conservan el ancho natural y reparten el sobrante; el mástil no escala. La composición requiere soporte CSS subgrid y se ha comprobado en Chromium 153. Una fila con poco contenido puede tener cajas más anchas que una llena; no cambia el tamaño de papel. La medición usa el mismo renderizador, sin ese crecimiento exterior, de forma conservadora.

Textos largos ajustan su altura. Si una cabecera o una pieza no cabe ni sola por ancho o alto, `collectProblems` recoge el fallo y bloquea Exportar e Imprimir. No se cortan notas ni se alteran digitaciones para ajustar la hoja.

Cambios concurrentes de interfaz

`ui/paginator.js` agrupa cambios con un retardo de 80 ms y un número de generación. Solo publica el resultado de la generación vigente. El resultado contiene una copia de canción y su firma, junto al contexto y el plan medidos. Una carga tardía de fuentes invalida la medida.

Exportar espera `settled()` y comprueba `isCurrent` antes de utilizar el resultado. Si hay otra edición durante la espera, vuelve a obtener un resultado vigente, con un máximo de tres intentos en el controlador de exportación. El zoom, limitado al 10–150 %, escala el visor sin intervenir en el reparto.

Exportación y tratamiento de texto

El HTML autónomo se construye desde los nodos del renderizador compartido. Incluye estilos y fuentes en línea, sin módulos JavaScript ni peticiones externas. No incluye controles de edición ni contornos de selección. El documento exportado sirve para ver e imprimir; no reemplaza al JSON editable.

Los textos de usuario se introducen mediante APIs de texto del DOM. Al ensamblar el documento se escapa el título HTML, se neutralizan cierres de `style` dentro del CSS y se rechazan etiquetas de ejecución o incrustación no permitidas en el cuerpo serializado. Los archivos JSON se validan antes de aplicarse.

Imprimir usa el mismo HTML en un marco y llama a la impresión del navegador. No hay un generador PDF nativo en el producto. El usuario elige A4 horizontal, escala 100 %, sin márgenes añadidos ni encabezados o pies del navegador. Las opciones concretas dependen de ese navegador y de la impresora.

Servidor y lanzador local

`scripts/serve.mjs` usa `node:http` y escucha exclusivamente en `127.0.0.1`, puerto 5174 por defecto. Admite `--port` con un entero entre 1 y 65535. Sirve archivos de la raíz del repositorio, resuelve directorios a `index.html`, asigna MIME por extensión y envía `Cache-Control: no-store`.

Solo admite GET y HEAD. Sus respuestas incluyen 400 para URL mal formada, 403 para una ruta rechazada por la comprobación de confinamiento, 404 para archivo ausente, 405 para método no permitido y 500 para otros errores de lectura. No es un servicio de publicación en red ni una API de datos de canciones.

`Abrir-Compas.cmd` se sitúa en su propia carpeta, comprueba `node.exe` y su versión y ejecuta `scripts/launch.mjs` sin depender de `npm.ps1`. Git conserva CRLF para los `.cmd` mediante `.gitattributes`.

El lanzador sondea `/package.json` en 5174. Si reconoce `name: compas-a-compas`, abre esa instancia; si detecta otro servicio o una respuesta inesperada, informa y no termina procesos ajenos. Si el puerto está libre, arranca el servidor y espera hasta 10 segundos. En Windows abre el navegador mediante `rundll32.exe url.dll,FileProtocolHandler`; si la apertura falla, muestra la URL y conserva el servidor propio.

La consola propietaria mantiene el proceso. Sus manejadores de cierre por señales limpian el hijo que creó. Una segunda ejecución que reutiliza el servicio no se convierte en propietaria. Puede reutilizar una copia del mismo producto arrancada desde otra carpeta; por ello, una actualización debe detener la instancia anterior antes de abrir la carpeta principal.

Claude Code, el conector MCP y Cloudflare son herramientas externas de desarrollo. No participan en el arranque ni en el uso de la aplicación.

Comprobaciones y límites de la evidencia

La evidencia de cada versión permanece identificada en la tabla y en `docs/ESTADO.md`. La ampliación 0.5.0 se ha comprobado en Chromium 153 sobre Linux; las pruebas Node se ejecutan también en el PC con Windows mediante el conector.

Comprobación registrada	Resultado y entorno
Pruebas Node, 0.5.0	171 superadas en Windows; incluye contexto tonal, validación, JSON e historial
Interfaz 0.5.0	Campos individuales y casilla, herencia, edición, reapertura y recuperación; sin scroll interno en 1440×900, 1366×768 y 1280×720 con el ejemplo; en 800×600 se conserva desplazamiento interno
Hoja 0.5.0	Títulos enmarcados y franjas fuera de la línea con textos de distinta altura; 40 posiciones, 24 compases, continuaciones, marcas, cuatro trastes y texto tratado como texto
Salida 0.5.0	HTML autónomo sin peticiones externas, color y etiquetas; PDF de 1 y 6 hojas A4 horizontal en Chromium sin interfaz
Esquema 0.5.0	1751 casos de paridad manual/AJV con los campos nuevos y paleta
Pruebas Node, 0.4.1	150 superadas en Windows mediante el conector
Interfaz 0.4.1	Secciones apiladas; formulario habitual sin scroll en 1440×900, 1366×768 y 1280×720. En 800×600 conserva scroll interno y todos los campos. Captura del PC sin errores JavaScript
Paleta 0.3.0	JSON, reapertura, grados naturales y alterados, etiquetas, contraste, restauración, exportación e impresión con estilos de impresión
Esquema 0.3.0	1685 casos de paridad entre validador manual y AJV con paleta
Interfaz 0.2.0	Seis tamaños de escritorio de 800×600 a 1440×900, móvil 390×844, JSON, menús, edición, 40 posiciones y 30 compases

Comprobación registrada	Resultado y entorno
Marcas por compás, 0.2.0	Contexto inicial, cambio de compás, vista global, edición de tramos desde su interior, destinos y marcas guardadas sin desplazamientos
HTML exportado, 0.2.0	Ejemplo idéntico byte a byte al de 0.1.3
Validación de esquema	1575 casos de paridad con AJV registrados durante las fases; esquema sin cambio
Chromium 153 en Linux, 0.1.3	JSON de 19 compases y 36 posiciones en dos hojas; guardado idéntico y medidas de vista previa/exportación iguales
Geometría y continuaciones, 0.1.3	Crecimiento a 4 y 8 trastes; 30 posiciones con continuación y barras sin duplicados
Desplazamiento adaptable	Ocho tamaños de 360 a 1440 px en 0.1.2; estilos de desplazamiento sin cambio en 0.1.3
PDF en navegador sin interfaz	1, 2, 3 y 6 páginas en fases anteriores; papel A4 horizontal y sin página vacía final
Lanzador	Servidor real, reutilización y cierre en Linux; apertura de navegador Windows mediante simulación

Quedan pendientes el doble clic y cierre de consola reales en Windows, el diálogo de impresión nativo y las comprobaciones en Firefox y Safari. No se declara una auditoría completa de accesibilidad ni una prueba en dispositivos móviles físicos.

La paginación mide contenido en el DOM; canciones extensas o textos muy largos pueden aumentar el tiempo de reparto. No se han definido presupuestos de rendimiento ni límites de tamaño de archivo. La recuperación depende de la disponibilidad y cuota de `localStorage`.

Mantenimiento y documentación

Para comprobar el producto se ejecuta `npm test` desde la raíz. Para abrirlo se usa el lanzador o `npm start`. No se requiere `npm install`.

Los manuales canónicos son `docs/manuals/USO.md`, `docs/manuals/INSTALACION.md` y este DTS. Sus PDF, LaTeX y manifiestos SHA-256 están en `docs/latex`. `scripts/build_docs.py` los construye con Python 3.10 o superior, Pandoc y LuaLaTeX o XeLaTeX. Estas herramientas son solo de documentación.

Al modificar el modelo, revisar el esquema, el validador, la serialización y sus pruebas; una versión de esquema nueva requiere una política explícita de compatibilidad. Al modificar geometría o estilos de hoja, revisar vista previa, HTML, continuaciones y PDF sobre el mismo conjunto de datos. No se modifica `references/` para forzar una comparación visual.

Registrar versión y entorno de cada comprobación en `docs/ESTADO.md`. Actualizar desde `main` con `git pull --ff-only`, conservando los cambios locales. Los manuales parten de la carpeta principal.